Entrega final de la solución del reto (EKF with Puzzlebot)

Mendoza, Daniel. Morales, Gael.
ITESM CEM
{A01747817, A01754461}@tec.mx

***Abstract*— This document focuses on the development of a differential drive robot capable of localizing itself in an open environment, avoiding obstacles, and reaching a defined target. The robot relies on various vision-based sensors (ArUco markers) and odometry to support autonomous navigation. It operates within the ROS2 framework, where dedicated nodes manage motion control, sensor integration, and localization. Simulation was conducted in Gazebo to validate the robot's behavior. A hybrid navigation strategy combining Bug0 and Bug2 algorithms was implemented, enabling dynamic switching based on environmental conditions. To enhance localization accuracy, an Extended Kalman Filter (EKF) was used to predict and correct the robot's position. The system was evaluated in scenarios with varying marker visibility, demonstrating robust navigation and localization performance.**

NOMENCLATURA

- $r = radio\ de\ la\ rueda$
- $l = distancia\ entre\ ejes\ (track)$
- $\omega r = velocidad\ angular\ de\ la\ rueda\ derecha$
- $\omega l = velocidad\ angular\ de\ la\ rueda\ izquierda$
- $v = velocidad\ lineal$
- $\omega = velocidad\ angular$
- $Kv = Ganancia\ de\ velocidad\ lineal$
- $K\omega = Ganancia\ de\ velocidad\ angular$
- $K_{fw} = Ganancia\ follow\ wall$
- $S = Vector\ de\ estado\ estimado$
- $H_k = Jacobiano\ del\ modelo\ con\ respecto\ al\ estado.$
- $\Sigma_k = Matriz\ de\ covarianza$
- $u = Entradas\ al\ sistema\ (encoders)$
- $F_k = Jacobiano\ de\ movimiento\ con\ respecto\ a\ entradas.$

I. PLANTEAMIENTO DEL PROBLEMA

En el desarrollo de robots móviles autónomos, la estimación precisa de la posición es un componente crítico para una navegación eficiente y segura. El uso de sensores como los encoders suele considerarse confiable, en la práctica estos dispositivos presentan errores acumulativos debidos al deslizamiento de ruedas, irregularidades en el terreno y otras fuentes de ruido. En el caso del "puzzlebot", el cual es un robot differential drive que utiliza dos motores DC con un encoder y en donde es necesario utilizar la odometría para estimar su posición.

La robótica móvil cada día tiene un mayor peso en las industrias, que van desde la paquetería y limpieza del hogar hasta la industrialización como la logística o manufactura. Muchos de estos robots necesitan llevar cosas de un lugar a otro o tomar objetos, por lo que deben de tener una precisión milimétrica con la finalidad de evitar tener errores que pueden ser incluso fatales. Por ello es importante tomar en cuenta el ruido acumulado y cuanto afecta al sistema del robot en el cual estamos operando o trabajando.

El hecho de tener diferentes sensores permite que el sistema de localización sea más robusto, ya que el uso de cámaras y LiDAR nos permite visualizar de una manera más precisa el entorno en donde se encuentra el robot y con ello utilizar técnicas de localización de la posición del robot en el momento actual.

Para ello es necesario implementar métodos como el Extended Kalman Filter (EKF), el cual nos permite predecir el comportamiento del robot con base en sensores como los encoders de los motores que generan el movimiento del robot. También podemos corregir la predicción creada con base en la visión del robot, como el uso de la cámara y la detección de ArUco markers para determinar la posición exacta del robot en ciertos puntos de la navegación. Por último la navegación es un punto importante al momento de evitar obstáculos y por ello el uso de algoritmos que permitan realizar estas maniobras son indispensables, en este caso el uso de bug0 y bug2 es una opción viable y por lo tanto el combinar ambos es posible para evitar obstáculos de gran tamaño o dificultad.

II. MARCO TEÓRICO

Para comprender el contexto del proyecto debemos de comenzar conociendo un poco acerca del tipo de robot móvil **differential drive** que se está utilizando, su nombre es puzzlebot. El puzzlebot es un tipo de robot terrestre que se desplaza gracias a dos ruedas ubicadas en ambos lados del chasis y controladas de forma independiente. Por lo que

el robot puede avanzar en línea recta y dar vueltas de forma fácil variando la velocidad de cada rueda. Su modelo cinemático está expresado de la siguiente manera:

$$v = \frac{r(\omega_r+\omega_l)}{2} \ , \ \omega = \frac{r(\omega_r-\omega_l)}{l}$$

Mientras en el plano, el movimiento del robot puede modelarse de una forma definida como:

$$x' = v \cdot cos(\theta)$$
$$y' = v \cdot sin(\theta)$$
$$\theta' = \omega$$

Otro concepto que es indispensable conocer es el término de **odometría**, la cual es un método que estima la posición y orientación de un robot a lo largo del tiempo en donde es necesario utilizar datos de sensores para poder determinar esta posición y orientación. Por lo general en los robots differential drive los sensores usados son encoders en las ruedas, los cuales ayudan para medir velocidades. La odometría es muy útil sobre todo porque no es necesario el uso de sensores de alto costo o que requieran de un procesamiento de datos extenso, es un sistema simple y de bajo costo, pero genera un problema de medición ya que el error acumulado es muy grande conforme pasa el tiempo debido a imperfecciones en el camino, deslizamiento de las ruedas e incluso ruido del mismo encoder y por ello es necesario combinarlo con métodos de localización más robustos.

El filtro de Kalman es un algoritmo que permite estimar un estado a partir de mediciones con ruido (perfecto para el uso con encoders que generan ruido). En el caso del uso del puzzlebot su uso es excelente para generar predicciones con base en la odometría. Por el otro lado un filtro de Kalman extendido (EKF) es una extensión para sistemas no lineales como el robot differential drive, en donde linealiza el modelo con la finalidad de generar una predicción y una corrección. En el caso de este proyecto se usa una cámara y se tiene el conocimiento de la posición de un ArUco, el cual si se detecta realizará una corrección de la posición que fue predecida por el filtro de Kalman, creando un filtro de Kalman extendido.

III. ALGORITMO DE NAVEGACIÓN REACTIVA

Los algoritmos de bug son estrategias de navegación reactiva que tienen como objetivo llegar a una meta mientras se evitan obstáculos. Son útiles para entornos desconocidos de los cuales no se tiene un mapa y son funcionales con sensores como el LiDAR. Existen diferentes tipos de bug, sin embargo para el planteamiento de una fusión entre bug0 y bug2 denominada bug3 o bug híbrido es necesario conocer los principios de funcionamiento de estos algoritmos.

A. Funcionamiento Bug0

Su objetivo principal es llegar al objetivo, si un obstáculo es detectado entonces el robot rodeará al obstáculo (como una pared) hasta dejar de detectarla y llegar al objetivo. Se puede definir de qué lado se desea que se rodee el objeto, si se quiere rodear por el lado izquierdo o derecho.

B. Funcionamiento Bug2

Este algoritmo genera una línea que va desde el punto de partida hasta el objetivo y el robot sólo es desviado cuando hay un obstáculo que bloquea la línea, por lo que el robot rodea el objeto hasta volver a encontrar la línea y volver al objetivo. Lo cual genera trayectorias eficientes.

C. Funcionamiento Bug Híbrido (Bug3)

Ambos de los bug anteriores (bug0 y bug2) tienden a tener problemas en algunas situaciones concretas, por ello es que se tomó la decisión de incorporar una mejora, ya que en situaciones como un cuarto cuadrado bug0 tiende a perderse y quedar en un bucle, lo cual no ocurre si usamos bug2. Por el otro lado bug0 tiene un tiempo de reacción más rápida que bug2, por lo que se incorporan ambos de los bug y se usan dinámicamente dependiendo de la situación, en algunos casos se usa bug0 para seguir otra pared y si encuentra una m line regresa a ella cuando sea necesario.

IV. FILTRO DE KALMAN EXTENDIDO PARA LOCALIZACIÓN

El objetivo del filtro de kalman extendido para la localización del puzzlebot se basa en dos ideas diferentes, ya que se emplean diferentes sensores para lograr integrar este filtro. La primera idea genera una **predicción** usando odometría con encoders, la segunda idea genera una **corrección** de acuerdo con una detección visual de un ArUco en una posición conocida.

Primero es necesario generar una predicción y para ello es indispensable el conocer el vector de estado del puzzlebot y iniciar una matriz de covarianzas iniciales para poder representar la incertidumbre inicial del estado del robot. A continuación se encuentra la representación de ambas matrices:

$$s_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}$$

Imagen 1.1. Vector de estado estimado

$$\Sigma_k = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{x\theta} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{y\theta} \\ \sigma_{\theta x} & \sigma_{\theta y} & \sigma_{\theta\theta} \end{bmatrix}$$

Imagen 1.2. Matriz de covarianza.

También es posible encontrar las fórmulas de velocidad que se utiliza en un robot differential drive, tanto para una velocidad lineal v, como para una velocidad angular ω:

$$v = \frac{r(\omega_r+\omega_l)}{2} \ , \ \omega = \frac{r(\omega_r-\omega_l)}{l}$$

Con las fórmulas para la velocidad lineal, angular y el vector de estado del puzzlebot podemos encontrar el modelo de movimiento y las entradas de velocidades.

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + v * \Delta t * cos(\theta_k) \\ y_k + v * \Delta t * sin(\theta_k) \\ \theta_k + \omega * \Delta t \end{bmatrix}$$

Imagen 1.3 Modelo de transición del estado

$$u = \begin{bmatrix} \omega_r \\ \omega_l \end{bmatrix}$$

Imagen 1.4 Entradas al sistema por parte de los encoders.

Una vez se tiene este modelo de movimiento de un robot differential drive podemos continuar con la primera etapa para el desarrollo de un filtro de Kalman extendido, para ello es necesario el establecer el valor de Δt. Ahora que sabemos el modelo de movimiento del puzzlebot podemos determinar que el modelo de movimiento es no lineal debido a que depende del ángulo de orientación del robot. Por lo tanto si es necesario aplicar el filtro de Kalman se debe linealizar y por ello es que usamos la serie de Taylor, además de ello el modelo no lineal se aproxima más a la serie de Taylor de primer orden. De acuerdo con esto podemos encontrar el jacobiano del modelo respecto a nuestro estado, una matriz que es denominada como H_k y está compuesta por la derivada parcial del modelo de transición del estado respecto a $s_k = [x_k, y_k, \theta_k]$, lo cual si se resuelve nos deja con una matriz 3x3 que modela la matriz de transición linealizada del sistema:

$$H_k = \frac{\partial}{\partial s^T} h(s) = \begin{bmatrix} 1 & 0 & -T * v * sin(\theta) \\ 0 & 1 & T * v * cos(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

Imagen 1.5. Jacobiano del modelo respecto al estado

Posterior a encontrar nuestro modelo jacobiano respecto al estado podemos encontrar el jacobiano del modelo respecto a las entradas, esto es necesario porque este jacobiano nos indica cómo afectan los cambios en las entradas de u (Imagen 1.4) al estado. Al saber que el estado en movimiento sigue siendo no lineal se debe de aplicar la serie de Taylor, por lo que se debe generar una derivada parcial del estado en movimiento respecto a $u = [\omega_r\ \omega_l]$.

$$F_k = \frac{\partial}{\partial u^T} h(s, u) = \frac{1}{2} rT \begin{bmatrix} cos(\theta) & cos(\theta) \\ sin(\theta) & sin(\theta) \\ \frac{2}{L} & -\frac{2}{L} \end{bmatrix}$$

Imagen 1.6. Jacobiano de movimiento con respecto a entradas.

De igual manera se debe de considerar una matriz de covarianza de ruido, la cual es necesaria para modelar la predicción, esta matriz es denominada como L_k.

$$L_k = \begin{bmatrix} Kr|\omega_r| & 0 \\ 0 & Kl|\omega_l| \end{bmatrix} = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_l^2 \end{bmatrix}$$

Imagen 1.7. Matriz L_k de covarianza de ruido.

Con estas matrices es posible multiplicarlas para generar una predicción con base en la matriz de covarianza de la imagen 1.2, ya que las velocidades irán cambiando y los cálculos generarán una predicción de en donde puede encontrarse el puzzlebot. Al final la predicción se calcula de la siguiente manera:

$$\Sigma_k = H_k * \Sigma_{k-1} * H_k^T + F_k * L_k * F_k^T$$

Una vez encontrada la predicción ya tenemos la primera parte para generar un filtro de Kalman extendido (EKF), ahora es cuando debemos de pasar a la etapa de **corrección** utilizando un sensor que pueda ayudar a corroborar la posición del puzzlebot. En este caso el puzzlebot tiene implementada una cámara en el frente, con la cual es posible realizar una detección de ArUco, el cual se encuentra fijo en una posición conocida, con la cual podemos corregir la posición y reducir el valor de Σ_k.

Ahora para la generación de un modelo de observación es necesario determinar las salidas que debemos de observar. En este caso estamos usando una cámara que detecta ArUco a una distancia determinada y en un ángulo determinado ya que los ArUco tienen posiciones y ángulos conocidos, por lo que es posible encontrar la posición y ángulo relativo del robot hacia el ArUco. Esto se puede ver si modelamos la distancia con el siguiente modelo de observación:

$$y_k = g(m_i, \mu_k) = \begin{bmatrix} \sqrt{\Delta x^2 + \Delta y^2} \\ atan2(\Delta y / \Delta x) - \theta_k \end{bmatrix}$$

Queremos aplicar el operador de esperanza matemática, por ello es que necesitamos una aproximación lineal de un sistema no lineal, por lo que debemos de aplicar serie de Taylor de primer orden, con respecto al observador que tenemos y_k. Lo cual deja que el jacobiano de la observación con respecto al estado quede de la siguiente forma.

$$G_k = \frac{\partial}{\partial s^T} g(m_i, s) =$$

$$\begin{bmatrix} \frac{-\Delta x}{\sqrt{\Delta x^2 + \Delta y^2}} & \frac{-\Delta y}{\sqrt{\Delta x^2 + \Delta y^2}} & 0 \\ \frac{\Delta y}{\Delta x^2 + \Delta y^2} & \frac{-\Delta x}{\Delta x^2 + \Delta y^2} & -1 \end{bmatrix}$$

También es necesario incorporar una matriz de covarianza representando el ruido que es presentado por la cámara que ve el ArUco, es una matriz similar a la vista en la Imagen 1.7 asociada con la distancia y el ángulo. La matriz es denominada R_k.

$$R_k = \begin{bmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\psi^2 \end{bmatrix}$$

Posterior a obtener esta matriz de ruido R_k, es posible calcular la ganancia de Kalman es un peso que se define para calcular cuanto debe de confiar el observador en la

medición externa comparándolo con la predicción realizada en la odometría.

$$K_k = \Sigma_k G_k^T (G_k \Sigma_k G_k^T + R_k)^{-1}$$

Tomando en cuenta el estado es necesario actualizarlo basado en la diferencia existente entre la observación y lo que se espera ver según la predicción realizada anteriormente. El estado se actualiza de la siguiente manera.

$$s_k = s_k + K_k(z_k - g(m_i, s_k))$$

También es necesario sobreescribir la matriz de covarianza, ya que al momento de detectar una posición conocida por el puzzlebot se sabe en donde se encuentra, por lo tanto la matriz de covarianza tiene que reducirse, al igual que la elipse de confianza.

$$\Sigma_k = (I - K_k G_k)\Sigma_k$$

El sobreescribir la matriz de covarianza genera que la incertidumbre del sistema se reduzca drásticamente gracias a la corrección realizada. De ahora en adelante, después de haber realizado la corrección se adaptan los valores y se vuelve a iniciar la predicción hasta que la cámara detecte otro ArUco y la corrección sea realizada.

V. LÓGICA DE PROGRAMACIÓN

A. Nodo de trayectoria

El nodo de trayectoria, llamado también trajectory_node, tiene como principal objetivo el establecer puntos de forma secuencial como una referencia hacia donde queremos llegar con el robot. Se mantiene un set de coordenadas específicas X y Y que corresponden al objetivo deseado, además de que el nodo genera un atraso de dos segundos para esperar a la carga de RVIZ, Gazebo y los otros nodos se encuentren listos para recibir el mensaje.

El nodo publica un tipo de mensaje Pose en el tópico denominado /target, al cual se suscriben otros nodos como el nodo de navegación reactiva (sección ***B. Nodo de bug***), cada que el nodo de navegación reactiva termina de alcanzar el punto determinado debe de envíar por medio del tópico /next_point un mensaje de tipo Bool True, el cual determina que se ha llegado al objetivo y que se debe de pasar al siguiente punto de referencia, por ello es que el nodo de trayectoria responde a /next_point incrementando un índice interno y publicando el siguiente punto hasta que se terminen los puntos en la lista.

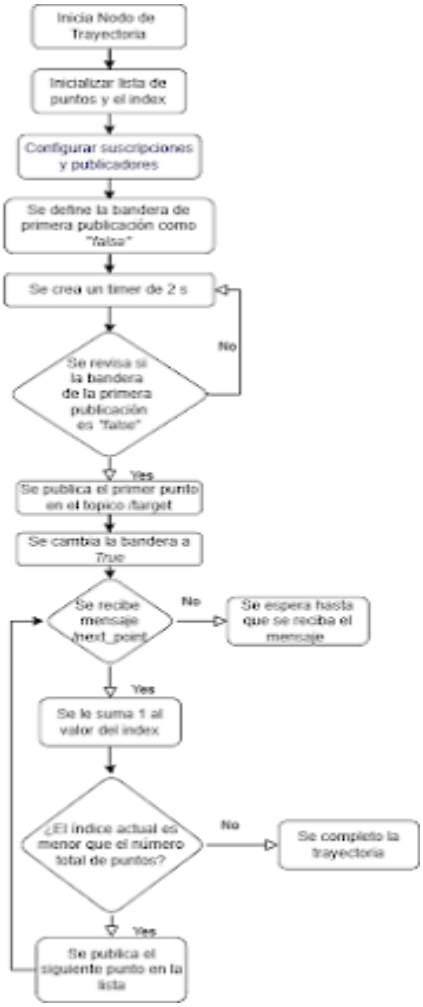


Imagen 1. Diagrama de flujo del nodo de trayectoria

B. Nodo de bug

El nodo de navegación reactiva tiene como objetivo el generar una ruta alternativa al objetivo si es que en su camino hay obstáculos que le impidan llegar. Esta navegación reactiva funciona gracias a la información de la odometría (sección ***C. Nodo de odometría EKF***) y los datos obtenidos de un LiDAR 360°. El objetivo es planteado por el nodo de trayectoria (sección ***A. Nodo de trayectoria***).

1) Publicadores.

El nodo tiene dos publicadores, el primero es denominado /cmd_vel, el cual transmite mensajes de tipo Twist y permite emitir velocidades lineales y angulares al puzzlebot para controlar su movimiento. Por otro lado el tópico /next_point es un tópico de tipo Bool, su función es actuar como "bandera" para poder indicar al nodo de trayectoria que se ha llegado al punto deseado.

2) Suscripciones

El nodo también cuenta con tres suscripciones, la primera suscripción es al tópico /odom, el cual envía mensajes tipo Odometry, en donde se especifica el valor que tiene el valor en la posición X, Y y el cuaternión. La segunda suscripción es al tópico /target, el cual es un mensaje tipo Pose enviado por el nodo de trayectoria, en este mensaje se reciben diferentes valores como el valor en metros de {X,

Y, $\theta, v, \omega\}$ en donde los valores importantes son el valor de X y Y, ya que indican la posición deseada. La tercera suscripción pertenece a /scan, el cual es un mensaje de tipo LaserScan y regresa un valor que va de 0 a 360 (el valor dependerá del tipo de LiDAR utilizado), dependiendo del valor dado por /scan se puede estimar si existe uno o múltiples objetos en los alrededores.

3) Timer

Dentro del nodo también es necesario el uso de un timer para el uso de la máquina de estados, el cual se encuentra inicializado en 0.1s y ejecuta la función de máquina de estados 10 veces por segundo.

4) Variables Iniciales

Se incluyen también una serie de variables globales que ayudan a realizar los cálculos necesarios para poder evitar obstáculos y llegar al objetivo, se definen datos como $Kv = 0.2$, mientras que $K\omega = 0.5$, estas ganancias fueron obtenidas de forma heurística haciendo pruebas con paredes dentro de gazebo y determinando si la llegada al punto genera aceleraciones y giros agresivos.

5) Funciones para extraer datos de los mensajes

En el nodo también se generan diferentes funciones para que la máquina de estados funcione. Existen tres funciones que dependen de los valores de las suscripciones y generan variables para generar un evasor de obstáculos, la **primera** función es odom_callback la cual tiene como objetivo extraer del mensaje el valor e X, Y y el cuaternión, el cuaternión es transformado a un ángulo yaw que nos permite conocer el heading del puzzlebot y por lo tanto ya tenemos la pose completa. La **segunda** función es el lidar_callback el cual se encarga de procesar los datos que vienen del lidar, con esa información se crea un diccionario el cual define que dependiendo el valor que viene del datos es la posición en la cual está el obstáculo. La **tercera** función es el target_callback, en el cual se define un tipo de dato Pose en donde se extrae el valor de X y Y, se reinician los parámetros del bug y se cambia el estado del controlador a go_to_goal. Además se traza un vector inicial y guarda el punto de inicio.

6) Funciones auxiliares

compute_line(): Calcula el vector unitario desde la posición actual hacia la meta.

Es necesario calcular el vector unitario que apunta hacia la meta desde la posición actual del robot, por ello es que se utiliza la siguiente fórmula:

$\vec{m} = \frac{(dx,dy)}{\sqrt{dx^2 + dy^2}}$ en donde $dx = x_{goal} - x_{actual}$ y en donde $dy = y_{goal} - y_{actual}$

distance(a,b): Calcula la distancia euclidiana entre la posición deseada y la actual.

En esta función es necesario calcular la distancia euclidiana entre dos puntos, esta distancia está dada por la siguiente fórmula:

$$d = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

on_m_line(): Determina si el robot se encuentra sobre la línea que va hacia la meta.

Se usan las siguientes fórmulas para evaluar si el robot se encuentra sobre la línea m que apunta hacia la meta:

$$dx, dy = valores\ iniciales\ de\ m\ line$$
$$px, py = Posición\ actual - Punto\ de\ colisión$$
$$dot = dx * py - dy * px$$

at_goal(): Verifica si el robot ya llegó a la meta o si se encuentra cerca.

obstacle ahead(): Detecta si hay un objeto frente al robot y si se encuentra a una distancia considerable (se determinó una distancia prudente a 0.35m considerando dimensiones del puzzlebot y que algunas veces necesita realizar giros).

move Robot(v, ω): Publica los comandos de velocidad al puzzle bot.

stop robot(): Detiene al robot enviando velocidades nulas.

7) Función go to goal

Esta función calcula la distancia que existe entre la posición deseada y la posición actual de la siguiente manera:

$$dx, dy = x_{goal} - x_{actual}, y_{goal} - y_{actual}$$
$$\theta_{goal} = atan2(y_{goal} - y_{actual}, x_{goal} - x_{actual})$$

Posteriormente se aplica un control proporcional a la velocidad angular, el cual está dado por $K\omega = 0.5$ como fue definido en la sección **4) Variables Iniciales,** mientras que la velocidad lineal se define en 0.0 si el ángulo excede 0.2 radianes o 11.46°, sino, se toma el valor de Kv para establecer esa ganancia como velocidad lineal y posterior a ello se mandan esas velocidades a la función moveRobot().

8) Follow wall

Esta función de seguimiento de pared es activada cuando el robot detecta un obstáculo frente a él y no puede seguir directamente la línea de meta. Por lo que sigue el contorno del obstáculo hasta que el camino quede libre.

El método de la función follow_wall(self, direction) indica qué dirección puede tener dos opciones, puede ser "left" o "right". Posterior a esto se calcula un vector tangente a la pared usando las medidas de LiDAR. Las variables $r1$ y $r2$ representan las distancias medidas por el LiDAR y los ángulos $t1$ y $t2$ representan las direcciones orientadas al objeto ubicado. Dependiendo del lado $r1$ mide la distancia directa hacia la izquierda $\theta_1 = 90°$ (o -90° a la derecha) y $r2$ mide la distancia directa hacia la izquierda $\theta = 45°$ (o -45° a la derecha).

Se definen también los puntos en las coordenadas cartesianas calculadas de la siguiente manera:

$$P1x, P1y = (r1 * cos(\theta_1), r1 * sin(\theta_1))$$

$$\vec{U} = UxTan, UyTan = r2 * math.cos(t2) - P1x, r2 * math.sin(t2) - P1y$$

$$\overrightarrow{UTanN} = \left(\frac{\vec{U}}{\sqrt{(UxTan)^2 + (UyTan)^2}}\right)$$

Ahora se crea un vector perpendicular a la pared, pero para poder obtener esa proyección escalar se debe de realizar un producto punto con la fórmula a continuación:

$$dot = P1x * UxTanN + P1y * UyTanN$$
$$UxPer = P1x - dot * UxTanN$$
$$UyPer = P1y - dot * UyTanN$$
$$UxPerN = \left(\frac{UxPer}{\sqrt{(UxPer)^2+(UyPer)^2}}\right)$$
$$UyPerN = \left(\frac{UyPer}{\sqrt{(UxPer)^2+(UyPer)^2}}\right)$$

Después de establecer UxPerN y UyPerN definimos variables necesarias para poder controlar el seguimiento de la pared, variables como d_{wall} que establece una distancia de la pared, un parámetro de interpolación β y una ganancia K_{fw} que representa una ganancia follow wall.

El controlador follow wall trata de permanecer a una distancia dwall de la pared y por ello se debe de crear un vector de corrección separando la pared del puzzlebot sobre el vector perpendicular. Se aplica la siguiente ecuación:

$$ExPer = UxPer - d_{wall} * UxPerN$$
$$EyPer = UyPer - d_{wall} * UyPerN$$

Tanto ExPer como EyPer son las componentes del vector corregido. Por ello es necesario convertir al vector en un ángulo tanto para los vectores corregidos como los vectores tangentes.

$$\theta_{per} = atan2(EyPer, ExPer)$$
$$\theta_{tan} = atan2(UxTanN, UyTanN)$$

El parámetro de interpolación se introduce dentro de la siguiente fórmula en conjunto con los ángulos, dando prioridad al seguimiento de la dirección de la pared pero con la opción de permitir correcciones laterales.

$$\theta_{fw} = \beta \cdot \theta_{tan} + (1 - \beta) \cdot \theta_{per}$$

Posterior a esto se normaliza el ángulo, se fija una velocidad lineal de 0.10, mientras que el ángulo no pase de 0.1 radianes o 5.72°. Por el otro lado la ganancia $K_{fw} = 1.5$ es multiplicada por el ángulo normalizado para poder obtener la velocidad angular. Teniendo ambas velocidades (lineal y angular) es posible enviar mediante un mensaje de tipo Twist las velocidades al puzzlebot.

La ganancia K_{fw} fue elegida por medio de diferentes pruebas del método heurístico, se señalaron puntos al otro lado de una pared y era necesario determinar si el puzzlebot podía girar lo suficientemente rápido con la finalidad de evitar una colisión, por ello es que un K_{fw} de 1.5 es necesario para poder evitar una colisión y poder continuar con el seguimiento de pared.

9) Máquina de estados

El comportamiento general del robot se rige principalmente por una máquina de estados finita, que organiza y coordina las funciones que se ejecutan según las condiciones del entorno. De esta forma se permite que el puzzlebot funcione de manera reactiva ante obstáculos mientras sigue una trayectoria. Se encuentra compuesta de los siguientes estados:

9.1. stop_robot: Es el primer estado que se ejecuta, en donde se manda a llamar a la función stop_robot(), la cual envía velocidades de 0 al puzzlebot. Se activa si se llega al objetivo o cuando se inicia por primera vez el nodo.

9.2. Go_to_goal: Es el estado principal en el cual el robot se dirige a la meta en línea recta. Un control proporcional es empleado para únicamente avanzar hacia el objetivo hasta que algún objeto sea detectado. Si el puzzlebot se encuentra en el estado go_to_goal y la función at_goal() es True, entonces el estado go_to_goal reestablece las variables y pasa al siguiente punto.

9.2. avoid_obstacle: Es un estado intermedio que determina de qué lado es mejor evitar el obstáculo, por ello se usa la información del LiDAR. Aquí también es donde se emplea el follow_wall() y se encuentra constantemente evadiendo un objeto y buscando la m line para determinar si es posible ir directamente hacia el objetivo.

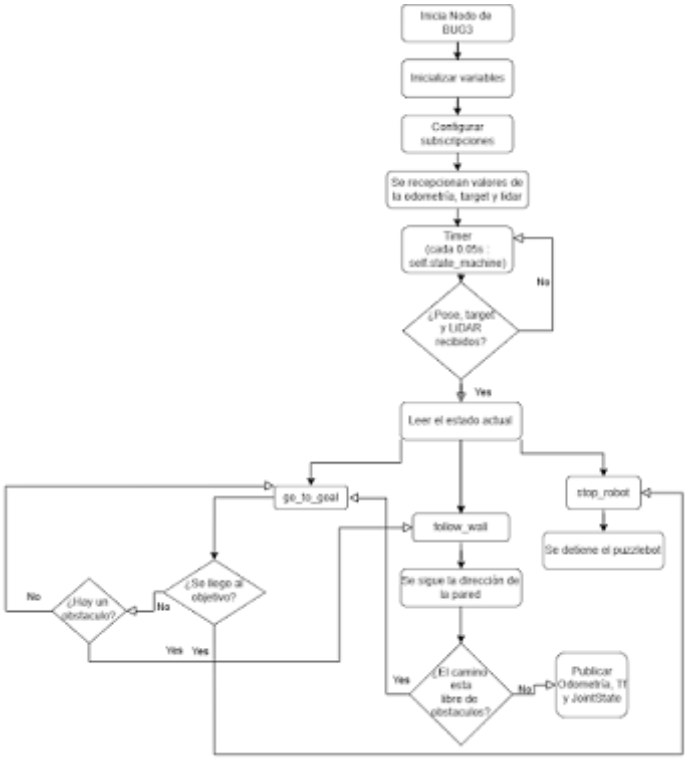


Imagen 2. Diagrama de flujo del nodo de bug

C. Nodo de odometría EKF

Este nodo tiene como objetivo estimar la posición y orientación del puzzlebot en un entorno de simulación utilizando un filtro de Kalman extendido que utiliza los datos provenientes de los encoders de los motores de las ruedas y la cámara y la detección de ArUcos.

1) Publicadores

Este nodo cuenta con tres publicadores:

1. "*/odom":* Se encarga de publicar mensajes del tipo Odometry, con la posicion y orientacion estimada de robot en el frame *world*, este tipo de mensaje igual tiene la covarianza de la estimación
2. *"/tf*": Se publica la transformación entre los frames *world y estimated/base_footprint*, lo que permite que se pueda ver la trayectoria del puzzlebot dentro

de RViz2.

3. *"joint_states":* Publica mensajes de tipo JointStates, actualizando el estado de las ruedas para que el modelo en RViz muestre correctamente las rotaciones de las llantas.

2) Suscripciones

Este nodo cuenta con los siguientes suscripciones a los siguientes tópicos:

1. *"/VelocityEncL"* y *"/VelocityEncR"*: Usa mensajes de tipo *Float32* que representa la velocidad angular de las ruedas. Estos valores obtenidos son utilizados para calcular la odometría.
2. *"/aruco_detections"*: Usa mensajes personalizados que contiene la posición relativa de los marcadores detectados desde la cámara del robot. Esta información se transforma a coordenadas globales y corrige la estimación de la pose.
3. *"clock"*: Proporciona el tiempo simulado.

3) Variables iniciales

Dentro de este nodo se definen las siguientes variables:

1. *"r=0.033"*: Radio de las ruedas del puzzlebot en metros. Esta medida es necesaria para convertir la velocidad angular y velocidad lineal de las ruedas.
2. "*l=0.160"*: Distancia entre las ruedas en metros.
3. "*x,y,theta=0.0,0.0,0.0"*: Representa la estimación inicial de la pose del robot en el plano, "x" y "y" son coordenadas cartesianas, mientras que theta es el ángulo de orientación (yaw)
4. *"P=np.eye(3)*0.001": En esta variable se inicializa la matriz de covarianza inicial del estado.*
5. *"R":* Es una matriz que describe la covarianza entre la distancia y el ángulo de la visión del ArUco.
6. "*vL,vR= 0.0 , 0.0":* Se asignan las velocidades angulares de las ruedas izquierdas y derecha a nulas para evitar movimientos:
7. "*I": es una matriz de identidad de 3x3*
8. *"MarkersPosition":* Se le asignan poses que tendrán los arucos dependiendo de su id para hacer la corrección adecuada.
9. Se define que el mensaje de odometría está referenciado al frame de "world" y tiene un frame hijo "estimated/base_footprint".
10. Se crea "self.tf_broadcaster", que es necesario para publicar las transformaciones entre frames de referencia.
11. Se define "self.js_msg" como un mensaje de tipo JointState, el cual es utilizado para publicar el estado estimado de las ruedas del puzzlebot
12. Se define la variable "self.markers" como una lista vacía que servirá para almacenar temporalmente los datos de los ArUcos detectados durante cada ciclo de ejecución del filtro.

4) Funciones

- **Encoder Callbacks:** Dentro de esta función se cuentan con callbacks que se ejecutan cada vez que se recibe un mensaje en este caso se ejecutan cada vez que se recibe un valor de la velocidad angular de las ruedas y se actualiza.
- **Clock_cb(msg):** Dentro de esta función se extrae el tiempo simulado y se guarda como el tiempo actual,
- **Aruco_cb(msg):** En esta función únicamente se almacena la lista de marcadores ArUcos detectados en la última iteración
- **Odometry callback:** Esta función se ejecuta periódicamente y tiene dos fases principales: predicción y corrección.
  Dentro de esta primera fase lo que se realiza es utilizar las velocid
  ades angulares de las ruedas para calcular el desplazamiento del robot durante el intervalo de tiempo transcurrido y con base al modelo cinemático diferencial, actualiza la pose estimada del puzzlebot y la incertidumbre asociada que en este caso es la matriz de covarianza, representando el error acomulado al desplazarse solo por odometria.
  En cuanto a la segunda fase se tiene la corrección del estado, en la cual si la cámara detecta algún ArUco con posición conocida, la función compara la posición estimada por la odometría con la observación realizada por la cámara y a través del uso del filtro de Kalman extendido, ajusta la estimación del estado y reduce la incertidumbre, corrigiendo el error acomulado.
  Finalmente la función pública el nuevo estado estimado y la matriz de covarianza en los tópicos correspondiente y permitiendo que otros nodos puedan acceder a la ubicación estimada por el nodo.

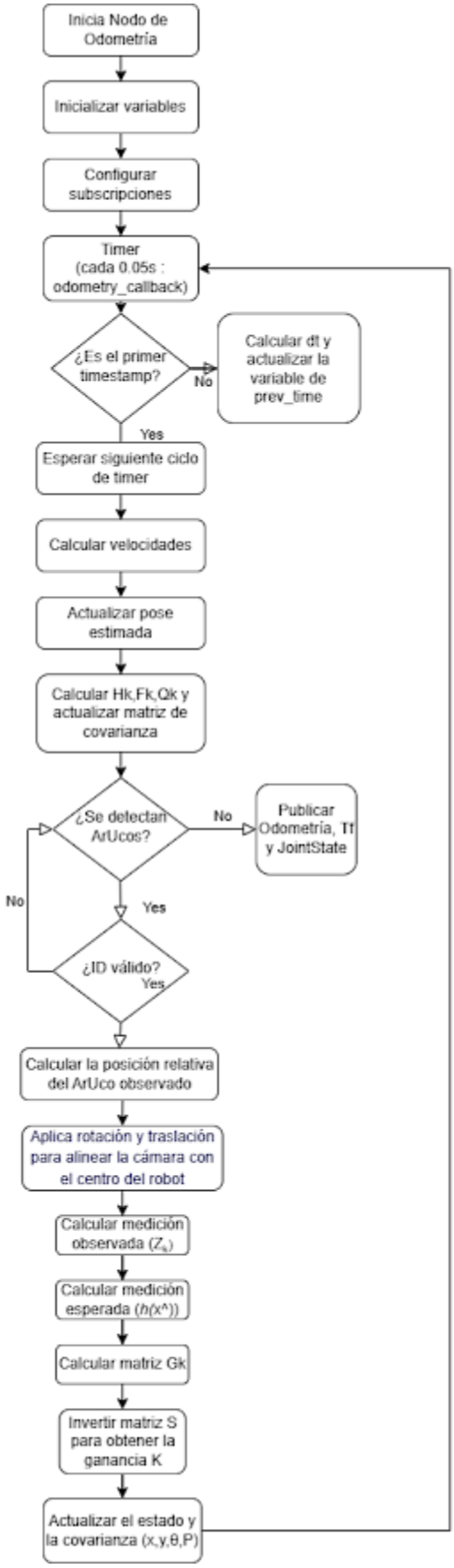


Imagen 3. Diagrama de flujo del nodo de Odometría.

D. Nodo de visión

El nodo de visión tiene como objetivo el detectar marcadores de tipo ArUco en la imagen proveniente de la cámara del puzzlebot y con esta imagen también estimar la pose (posicion y orientacion) respecto a la cámara del puzzlebot. Para poder realizar esto se utilizará la librería de OpenCV con el módulo de aruco y cv_bridge.

1) Publicadores

Dentro de este nodo se tienen 2 publicadores. El primer publicador que se tiene es el de *"/aruco_detections"*, el cual usa mensajes del tipo *MarkerPose*, el cual cuenta con un información del marcador ArUco detectado como el ID, su posición (x,y,z) y su orientación en cuaternión (x,y,z,w). Mientras que el segundo publicador es el *"camera_aruco"*, que usa un mensaje de tipo *"sensor_msgs/msg/Image"*, el cual contiene la imagen de la cámara del puzzlebot con los ejes 3D y el ID superpuesto que se colocan al observar un ArUco.

2) Suscripciones

Este nodo únicamente cuenta con un suscriptor el cual es se subscribe al tópico "*/camera"* y usa un mensaje de tipo "*sensor_msgs/msg/Image*". En el cual se tiene la imagen proveniente de la cámara del puzzlebot.

3) Variables iniciales

Se definen parámetros y variables iniciales. Las variables iniciales que se tienen son las siguientes:

- *fx,fy:* son las distancias obtenidas a partir del tamaño de la imagen y el campo de visión
- *cx,cy:* es el centro de la imagen
- *camera_matrix:* es la matriz 3x3 que se forma de la siguiente forma:

$$camera_matrix = \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix}$$

- *ancho(width):* se define con 640 píxeles
- *alto(height):* se define con 480 píxeles
- *marker_size:* es el tamaño físico del ArUco de 0.05m, para poder hacer la estimación de la profundidad.
- *min_distance:* es la distancia mínima para detectar el ArUco que es de 0.6 m.
- *max_distance:* es la distancia máxima para detectar el ArUco que es de 0.65 m.
- *max_angle_deg:*es el ángulo máximo para detectar correctamente el ArUco 35°.
- dictionary: en esta variable se define que se usará el diccionario de arucos 4X4_250

4) Función image_callback

La función image_callback es la función encargada de hacer el procesamiento de la imagen, esta se ejecuta cada vez que se recibe una nueva imagen desde el tópico de "*/camera"* y tiene como objetivo el detectar los ArUcos y estimar su pose (posición y orientación) con respecto a la cámara del Puzzlebot.

Lo primero que se realiza dentro de la función es

convertir la imagen de formato ROS a un formato necesario para que funcione correctamente con la librería OpenCV utilizando cvBridge. Posterior a esta adaptación de la imagen se utiliza la función "*cv2.aruco.detectMarkers()",* esta funcion regresa los valores de las coordenadas de los vértices de cada ArUco, así como los Id de cada uno y los marcadores que no cumplieron con los criterios definidos dentro de las variables iniciales.

Posterior a ello cuando se detecta un ArUco se utiliza la función de "*cv2.aruco.estimatePoseSingleMarkers()"*, la cual sirve para estimar la pose del marcador en términos del vector de rotación que sirve para describir cómo está orientado el ArUco con respecto a la cámara, así como también de los vectores de traslación que describe la posición del ArUco en el espacio con respecto a la cámara.

Una vez que se tienen los vectores de rotación y de traslación, se realiza un filtrado basado en la distancia y el ángulo para comparar si la distancia estimada del ArUco con respecto a la camara esta dentro del rango definido previamente en las variables iniciales del nodo, de igual manera se revisa si el ángulo de la orientación está dentro del umbral que se definió. Esto se realiza para poder tener certeza de que las detecciones poco confiables sean descartadas. En caso de que el ArUco cumpla con los criterios, se procede a calcular la orientación en forma de cuaternión para lo cual se convierte el vector de rotación en una matriz de rotación de 3x3 utilizando la función cv.Rodrigues(). Pero para que sea compatible con ROS2 se necesita que la matriz sea de 4x4, por lo que se construye esta matriz utilizando la función *"np.eye(4)",* la cual crea una matriz identidad de tamaño 4x4 en la cual se inserta la rotación en las primeras 3 filas y las 3 columnas. Esta matriz es importante ya que permite el transformar un sistema de coordenadas y para calcular el cuaternión utilizando la función "*tf_transformations.quaternion_from_matrix()".* Posterior a ello lo que se realiza es el crear el mensaje de tipo *MarkerPose*, en este mensaje se guarda el ID del ArUco junto con su pose. Una vez creado el mensaje se genera también un mensaje de tipo "*TransformStamped"* el cual tiene la finalidad de representar la pose del ArUco pero en forma de transformación entre 2 frames: el frame de la cámara (como referencia base) y el frame del ArUco. Ya teniendo este mensaje creado se publica mediante la función *"tf_boradcaster"* usando el método "*sendTransform".* Después se generan las anotaciones visuales sobre la imagen con el propósito de que cuando se suscriba la imagen procesada de este nodo sea más fácil la interpretación de los resultados, ya que se dibujaran ejes 3D del marcado sobre la imagen usando "*cv.aruco.drawAxis()"* y se escribe de igual forma el ID al centro del ArUco.

Este comportamiento al detectar un aruco se puede observar dentro de la siguiente imagen:

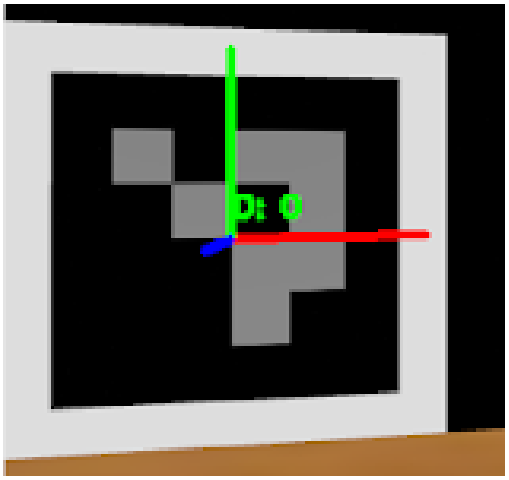


Imagen 4. Aruco detectado con ejes e ID

Finalmente, se publica el mensaje de "*ArucoDetection"* con la lista de los ArUcos detectados que son valido con su respectiva pose y de igual forma se publica la imagen procesada con sus anotaciones correspondientes y en caso de que se tenga un error durante los procesos se imprime un mensaje de error para facilitar la identificación de la falla.

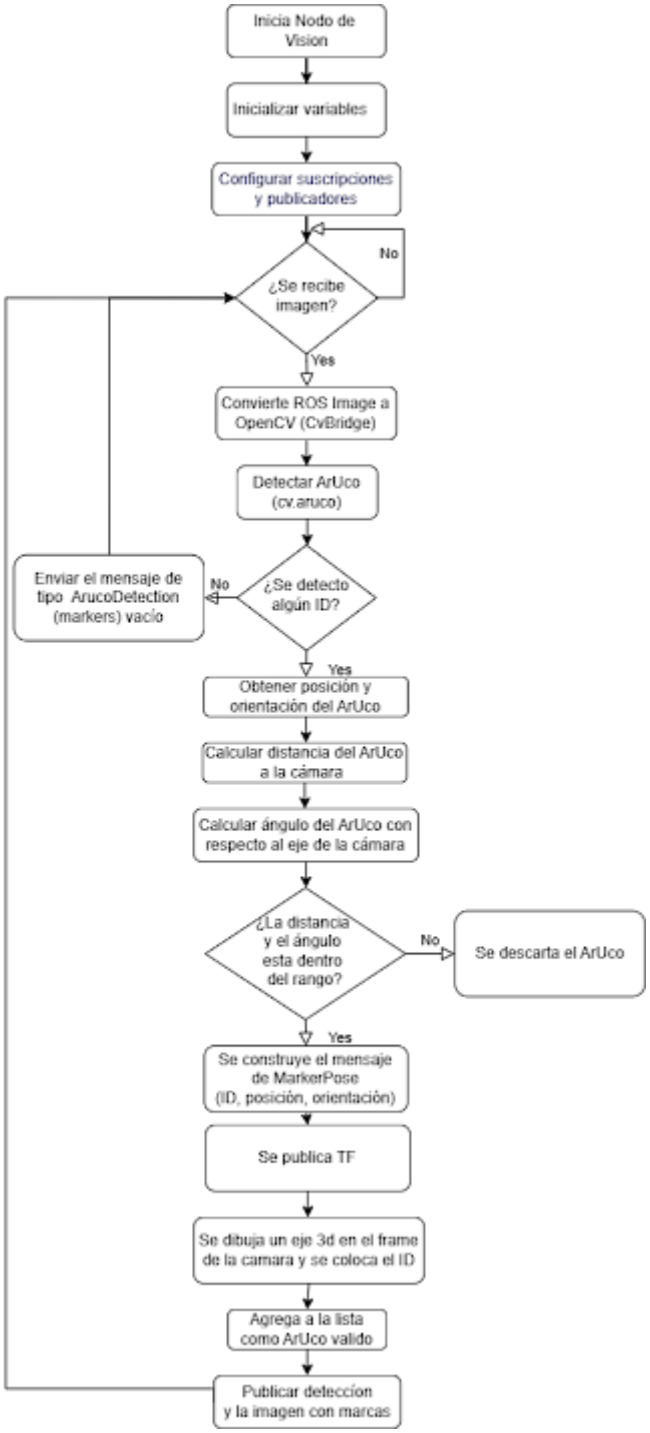


Imagen 5. Diagrama de flujo del nodo de Visión

E. Launch

Por parte de nuestro programa de launch, tiene como objetivo analizar y coordinar múltiples nodos relacionados con la simulación y visualización del comportamiento del Puzzlebot dentro del entorno de Gazebo y RViz2.

Lo primero que se realiza dentro de este programa es definir las rutas necesarias para algunos archivos que se utilizaran dentro del mismo. Se especifica cual es la ruta del archivo que contiene la configuración que se requiere para que RViz2 se pueda visualizar de manera adecuada, así como el archivo ".xacro" el cual define la descripción del robot en URDF. Posterior a ello se crearon 2 grupos en los cuales se realizan las siguientes acciones:

1) Grupo 1:

Este primer grupo corresponde al puzzlebot simulado en el cual se ejecutan los nodos explicados anteriormente dentro de este reporte que son: "trayectoria_puzzlebot","odom","vision" y "bug3". Cabe mencionar que se define que se usará el tiempo de simulación esto para evitar que en Gazebo y en RViz2 se tenga un comportamiento desfasado.

2) Grupo2:

Dentro de este segundo grupo se tiene un puzzlebot adicional al cual se le asigna un namespace llamado "*estimated/"*, por lo que los nodos estarán tendrán ese mismo namespace. Dentro de este grupo de igual forma se ejecuta el nodo de "*robot_state_publisher"* el cual tiene la función de publicar las transformaciones del puzzlebot a través del sistema TF2.

Finalmente lo que se realiza es el lanzar RViz2 con su correspondiente configuración y Gazebo, para que se pueda observar el comportamiento que tendrá el Puzzlebot.

VI. EXPERIMENTOS

Para cuantificar el desempeño del sistema de navegación autónoma, se desarrolló un nodo ROS2 adicional en Python que se suscribe tanto a la odometría estimada (/odom) como a la posición de referencia del entorno simulado (/ground_truth). El nodo registra y analiza métricas clave como el error de localización, la exactitud de llegada, la distancia total recorrida y las transiciones de estado dentro del algoritmo Bug3.

Cada experimento fue evaluado de acuerdo con:

- El **error promedio** acumulado en las coordenadas X, Y y θ.
- La **exactitud de llegada**, definida como la distancia euclidiana entre la posición final estimada y la posición objetivo.
- La **distancia total recorrida**, que puede interpretarse como un indicador de eficiencia en la planificación de trayectorias.

Se realizaron 4 experimentos en una trayectoria a los siguientes tres puntos: (-3.0, -1.9), (1.45, -1.9), (3.5, 0.0).

Esa trayectoria debe de ser completada en el **primer** experimento usando 5 ArUco alrededor del mapa que permiten corregir la pose. Posteriormente se realiza el **segundo** experimento, el cual elimina el primer ArUco, lo cual disminuye la cantidad de correcciones realizadas a la predicción de odometría. En el **tercer** experimento se eliminaron 2 ArUco reduciendo aún más la corrección. El **cuarto** es el experimento final en el cual no se presentó ninguna corrección, únicamente se realiza la predicción de la odometría y se estima la posición y dirección del puzzlebot.

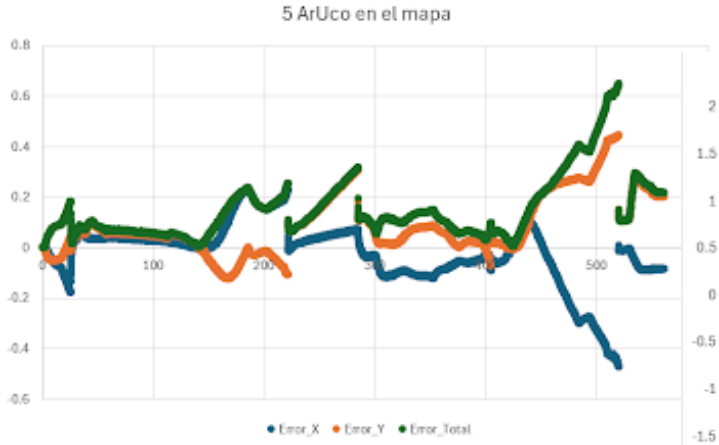


Imagen 6. Comportamiento de los errores de posición del puzzlebot en el tiempo.

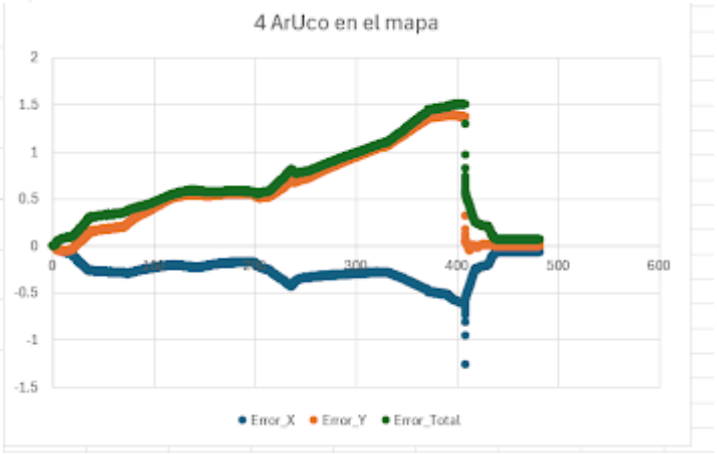
En la imagen 6 se puede observar el experimento realizado con 5 ArUco en el mapa, en donde se puede observar que el error se total (verde) va aumentando considerablemente y baja drásticamente a valores cercanos a cero, esto se le puede atribuir a la detección de arucos y la reducción del error gracias a la corrección realizada. sin embargo el error se ve disparado por eso de los 430 segundos, en donde se pasa mucho tiempo sin la detección de un aruco generando una acumulación de error que llega hasta 0.6486 metros o 64 centímetros con respecto al modelo original, posterior a esto es reducido hasta llegar a menos de 20 centímetros de distancia.

Error promedio X	-0.02575 metros
Error promedio Y	0.085886 metros
Error total promedio	0.1499 metros
Error máximo	0.6846 metros
Distancia total recorrida	17.25652 metros
Error de llegada	0.219082 metros

Tabla 1. Resultado de experimento con 5 ArUco.

Imagen 7. Comportamiento de los errores de posición del puzzlebot en el tiempo.

En la imagen 7 se puede observar el experimento realizado con 4 ArUco en el mapa, en este caso se puede ver un aumento de error en el comportamiento, ya que la falta de un solo ArUco generó inestabilidad en el comportamiento de puzzlebot y por lo tanto la estimación y el robot real se separaron generando una trayectoria diferente a la que fue propuesta en el nodo de trayectorias evitando algunos arucos, pero se puede observar que al final de la trayectoria el error disminuye al detectar un ArUco, esto se puede ver por eso de los 400 segundos.



Error promedio X	-0.26295 metros
Error promedio Y	0.560954 metros
Error total promedio	0.645975 metros
Error máximo	1.5055 metros
Distancia total recorrida	16.98507 metros
Error de llegada	0.068162 metros

Tabla 2. Resultado de experimento con 4 ArUco.

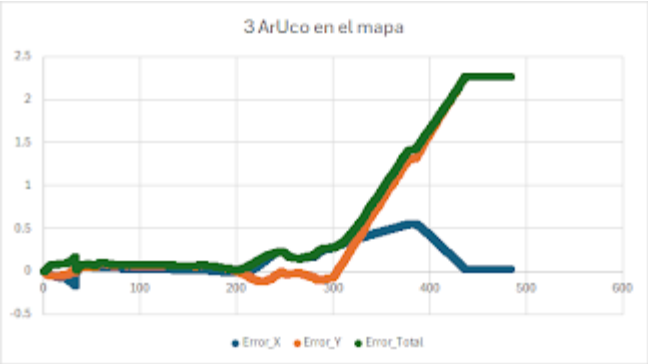


Imagen 8. Comportamiento de los errores de posición del puzzlebot en el tiempo.

En la imagen 8 se puede observar el comportamiento del puzzlebot en el tiempo, como se puede ver, se detectó un aruco en aproximadamente 20 segundos, en donde se mantuvo un error mínimo hasta aproximadamente los 200 segundos, después de ello el error comenzó a elevarse drásticamente hasta llegar a la “meta”, pero se puede observar que el error total es muy grande.

Error promedio X	0.149423 metros

Error promedio Y	0.558544 metros
Error total promedio	0.648609 metros
Error máximo	2.2593 metros
Distancia total recorrida	12.18866 metros
Error de llegada	2.259292 metros

Tabla 3. Resultado de experimento con 3 ArUco.

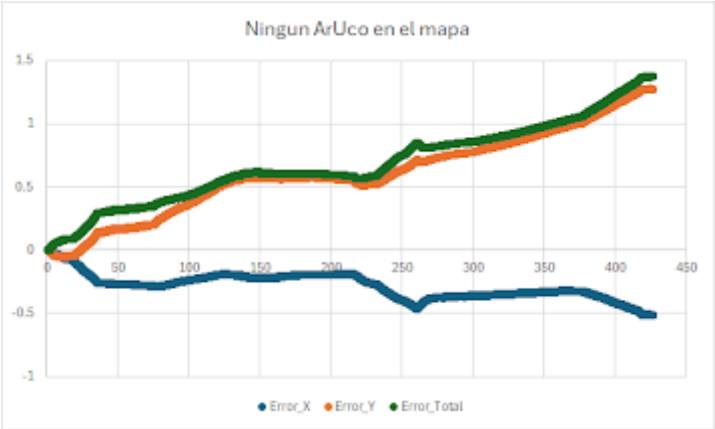


Imagen 9. Comportamiento de los errores de posición del puzzlebot en el tiempo.

En este último experimento se puede ver que no existe ninguna corrección en ninguna instancia del tiempo, por lo que inmediatamente se puede determinar que no hay un ArUco en el mapa que sea reconocido por la cámara y con el cual se realice una predicción. El error es grande y es claro cómo se acumula el error dentro de la odometría.

Error promedio X	-0.26295 metros
Error promedio Y	0.560954 metros
Error total promedio	0.645975 metros

Error máximo	1.5055 metros
Distancia total recorrida	16.98507 metros
Error de llegada	0.068162 metros

Tabla 4. Resultado del experimento con ningún ArUco.

VII. RESULTADOS

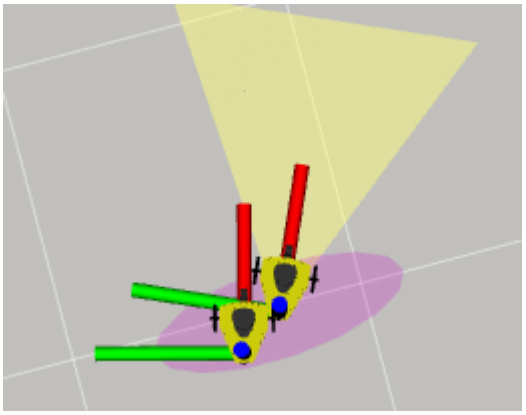


Imagen 10. Funcionamiento de la elipse y namespaces

Dentro de esta imagen se puede observar el uso de los namespaces ya que se tienen 2 puzzlebots dentro de RViz2.

De igual forma en la siguientes imágenes se puede observar el comportamiento de corrección por parte del filtro de kalman que tiene el sistema completo al detectar un ArUco.

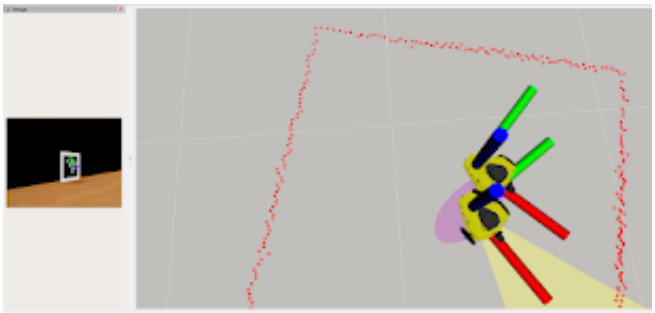


Imagen 11. Detección no válida del Aruco sin corrección

Dentro de esta primera imagen se puede observar como en la cámara se detecta un ArUco pero este no cumploe con los requisitos necesarios para que se pueda considerar como una lectura válida por ello se puede ver como la elipse de covarianza sigue creciendo aunque se haya detectado el ArUco y se puede ver como aun no se realiza alguna corrección a la pose .

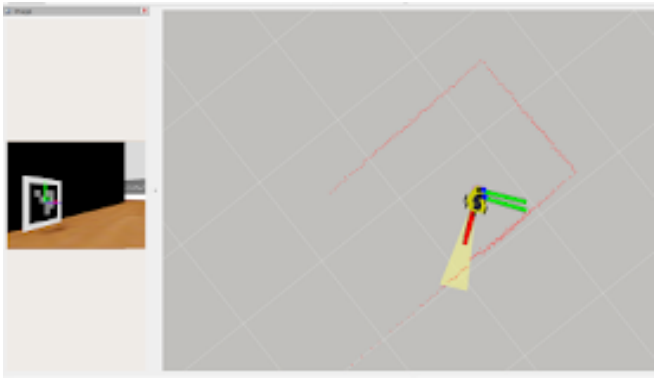


Imagen 12. Detección válida del Aruco con corrección

En esta segunda imagen se puede observar cómo se cumplieron los requisitos para que el ArUco pueda ser considerado como válido que es la distancia, ángulo e ID. Por ello se puede observar que la elipse de covarianza se reduce y se hace la corrección de la pose.

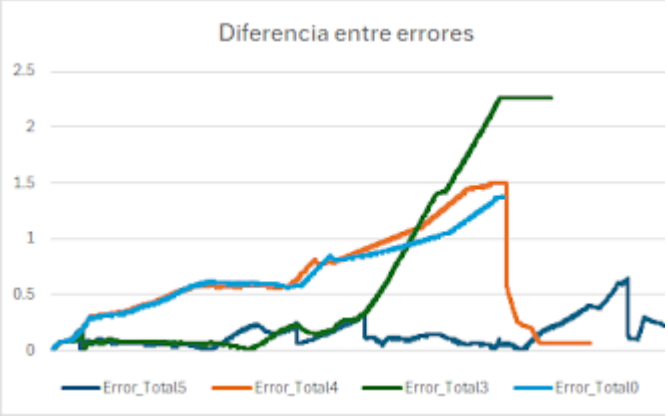


Imagen 13. Comparación de errores totales.

En la imagen 13 se puede observar que las pruebas tuvieron duraciones distintas, entre menor el número de ArUco se tiene una menor cantidad de duración y por lo tanto una menor cantidad de distancia recorrida.

También se puede observar que en el caso del error total con 3 ArUco tiene una mayor cantidad de error, es considerado que este error tiene que ver con la cantidad de ArUcos ignorados a lo largo del mapa afectó a la corrección del mismo y por ello el error fue acumulandose ya que un ángulo de giro mayor genera que ambas proyecciones (real y estimada) no sean paralelas y por lo tanto el error irá aumentando conforme al tiempo.

#ArUcos	5	4	3	0
Error promedio X	-0.025m	-0.262m	0.1494m	-0.262m
Error promedio Y	0.085m	0.560m	0.558m	0.561m
Error total promedio	0.149m	0.645m	0.648m	0.646m
Error máximo	0.684m	1.505m	2.259m	1.505m
Distancia total recorrida	17.25m	16.98m	12.188m	16.985m
Error de llegada	0.219m	0.068m	2.259m	0.068m

Tabla 5. Resultados de los 4 experimentos

Al analizar los resultados de los experimentos, se puede ver claramente que el mejor desempeño se obtiene cuando el sistema cuenta con 5 ArUcos distribuidos en el entorno. En este caso, tanto el error total promedio (0.149m) como el error máximo (0.684m) se mantienen bajos a lo largo de toda la trayectoria, y el error de llegada es de apenas 0.219m. Esto significa que el Puzzlebot es capaz de corregir constantemente su posición usando la información visual de los ArUcos, evitando que el error de la odometría se acumule y se descontrole.

Mientras que por otro lado, al ir bajando la cantidad de ArUcos el error empieza a aumentar de manera considerable, esto indica que el puzzlebot al pasar más tiempo sin detecciones de ArUcos hace que la incertidumbre y los errores de localización crezcan mucho más rápido.

Por lo que se puede se puede definir que entre más ArUcos se tengan dentro del ambiente simulado se tendrá una mayor precisión gracias a las correcciones del filtro de Kalman extendido.

Por último se puede observar dentro de las imágenes presentadas dentro de la sección de anexos, como se logró la creación de los nodos y sus tópicos y como se tiene establecida una comunicación entre sí. De igual forma se puede observar en la segunda imagen el árbol de transformadas que se obtuvo al usar el namespace de "*estimated*".

VIII. CONCLUSIONES

En conclusión, en este proyecto se pudo comprobar que implementar un filtro de Kalman extendido para la localización del Puzzlebot realmente mejora mucho la precisión del robot, sobre todo cuando se combina la odometría con la detección de ArUcos. Durante las pruebas, se vio claramente que tener más ArUcos distribuidos en el mapa ayuda a corregir los errores de la odometría y a mantener la estimación del robot cerca de la posición real. Por otro lado, cuando el robot solo depende de los encoders y no tiene correcciones visuales, el error crece bastante y la localización deja de ser confiable.

También, usar una estrategia híbrida de navegación, combinando los algoritmos Bug0 y Bug2, hizo que el robot pudiera adaptarse a distintos tipos de obstáculos y trayectorias, evitando que se quedara atorado y logrando que las rutas fueran más eficientes. En general, el sistema funciona bien, pero se notó que cuando pasa mucho tiempo sin ver un ArUco, la incertidumbre crece y se pierde precisión, así que sería bueno buscar maneras de siempre tener algún marcador visible o ajustar mejor los parámetros del filtro.

En resumen, el filtro de Kalman extendido con correcciones visuales y una navegación reactiva robusta hacen que el Puzzlebot sea capaz de moverse de forma autónoma y precisa.

REFERENCIAS

[1] *Navegación de robots móviles utilizando algoritmos Bugs Extendidos*. (s. f.). Rcs Ipn. Recuperado 30 de mayo de 2025, de https://rcs.cic.ipn.mx/2019_148_8/Navegacion%20de%20robots%20moviles%20utilizando%20algoritmos%20Bugs%20extendidos.pdf

[2] Ullah, I., Qian, S., Deng, Z., & Lee, J.-H. (2021). Extended Kalman Filter-based localization algorithm by edge computing in Wireless Sensor Networks. Digital Communications and Networks, 7(2), 187–195.1837–1838. https://www.sciencedirect.com/science/article/pii/S2352864820302601

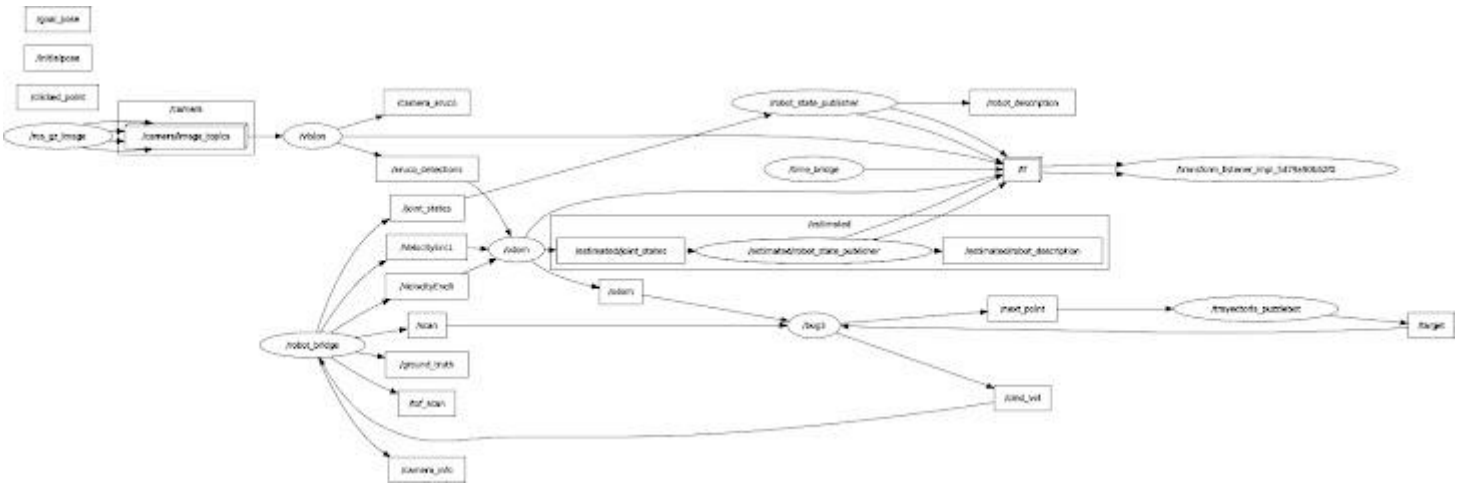
[3] Ribeiro, Maria & Ribeiro, Isabel. (2004). Kalman and Extended Kalman Filters: Concept, Derivation and Properties.

[4] OpenCV. (2025). Detection of ArUco Markers. OpenCV Documentation. https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

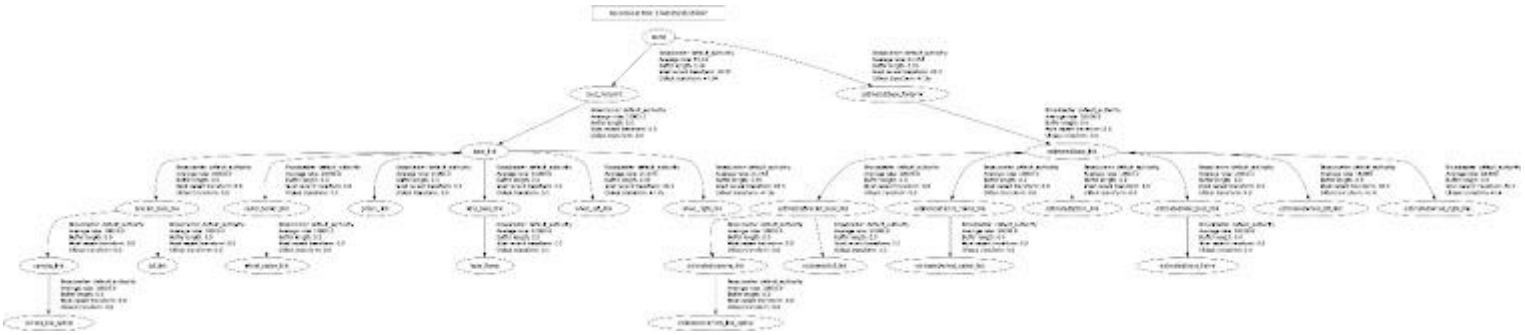
[5] *Differential Drive Kinematics*. (s. f.). Columbia. Recuperado 30 de mayo de 2025, de https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf

[6] Tran, H. T., Vo, T. C., Tran, D. L., Nguyen, Q. N. A., Ha, D. M., & Pham, Q. N. (2022). Extended Kalman Filter (EKF) Based Localization Algorithms for Mobile Robots Utilizing Vision and Odometry. En Proceedings of the 2022 IEEE 21st Mediterranean Electrotechnical Conference (MELECON) (pp. 1–6). IEEE. https://ieeexplore.ieee.org/document/9843066/authors#authors

ANEXOS



Anexo 1. Imagen del comportamiento de todos los nodos y sus tópicos



Anexo 2. Imagen del rqt_tf_tree del proyecto